

Introduction to Algorithms: 6.006（算法导论）

Massachusetts Institute of Technology（麻省理工学院） Instructors（授课教师）：Erik Demaine（埃里克·德梅因），Jason Ku（顾杰森），and Justin Solomon（贾斯汀·所罗门）

[session] Recitation 17（习题课 17）

MIT 6.006 Spring 2020, Recitation 17 · 主题：Dynamic Programming（动态规划）实战
—— Treasureship 与 Wafer Power · 对应 Lecture 15-17（DP Part 1-3）

[problem] Treasureship!（寻宝船！）

新桌游 Treasureship 的玩法是：在一个 $2 \times n$ 的矩形网格中放置 2×1 的 ships（船）。和普通 battleship（海战棋）一样，每艘 2×1 的船既可以水平放置也可以垂直放置，恰好占据 2 个网格格，且每个网格格只能被一艘船占据。每个网格格有一个正或负的整数值，表示在该格可以获得或损失多少 treasure（宝藏）。你可以在棋盘上放置任意多艘船，一个放置方案的总分是所有被船覆盖的网格格的值之和。设计一个高效的 dynamic-programming（动态规划）algorithm，确定一个能最大化你总分的放置方案。

[solution] Solution (解答) :

1. Subproblems (子问题)

- 游戏棋盘有 n 列、每列高度为 2 (等价地, 2 行、宽度为 n)
- 令 $v(x, y)$ 表示第 y 行第 x 列处的网格值, 其中 $y \in \{1, 2\}$, $x \in \{1, \dots, n\}$
- 猜测最优放置方案中如何覆盖最右边的方格
- 可以选择: – 不覆盖, – 垂直放置一艘船覆盖, 或 – 水平放置一艘船覆盖。
- 做出选择后, 棋盘的剩余部分可能不再是矩形
- 棋盘右侧呈现为下列情形之一:

```
(0) #####      (+1) #####      (-1) #####      (+2) #####      (-2)
###
      #####      #####      #####      #####
#####
      row 2
      row 1
```

(注: # 表示该格仍存在, 空白格表示已被吃掉/覆盖。(0) 两行等长; (+1) 第 2 行多 1 格; (-1) 第 2 行少 1 格; (+2) 第 2 行多 2 格; (-2) 第 2 行少 2 格。)

- 存在一个最优放置方案, 其中没有两艘船水平地上下对齐
- 证明: 改放两艘相邻的垂直船即可!
- 所以实际上只需要上面的前三种情形: 0、+1、-1
- 令 $s(i, j)$ 表示包含第 1 行第 1 到 i 列、第 2 行第 1 到 $i + j$ 列的游戏棋盘子集, 其中 $j \in \{0, +1, -1\}$
- $x(i, j)$: 只在棋盘子集 $s(i, j)$ 上放船时的最大得分
- 对 $i \in \{0, \dots, n\}$ 、 $j \in \{0, +1, -1\}$

2. Relate (关系)

- 若 $j = +1$, 可以用水平船覆盖最右边的方格, 或留空
- 若 $j = -1$, 可以用水平船覆盖最右边的方格, 或留空
- 若 $j = 0$, 可以用垂直船覆盖第 i 列, 或不覆盖最右方格之一

$$x(i, j) = \begin{cases} \max\{v(i, 1) + v(i - 1, 1) + x(i - 2, +1), x(i - 1, 0)\} & \text{if } j = - \\ \max\{v(i + 1, 2) + v(i, 2) + x(i, -1), x(i, 0)\} & \text{if } j = + \\ \max\{v(i, 1) + v(i, 2) + x(i - 1, 0), x(i, -1), x(i - 1, +1)\} & \text{if } j = 0 \end{cases}$$

3. Topo (拓扑序)

- 子问题 $x(i, j)$ 只依赖严格更小的 $2i + j$ ，所以无环。

4. Base (基本情况)

- $s(i, j)$ 包含 $2i + j$ 个网格格
- 若 $2i + j < 2$ ，则 $x(i, j) = 0$ (少于 2 格时无法放船!)

5. Original (原问题)

- 解是 $x(n, 0)$ ，即考虑全部网格格时的最大值。
- 存储 parent pointers (父指针) 以重构船的位置。

6. Time (时间)

- 子问题数: $O(n)$
- 每个子问题的工作量: $O(1)$
- $O(n)$ 运行时间

[note] Note (译者注)：这道题的精髓在于"非矩形棋盘怎么用矩形子问题表示"。第 1 行吃掉 i 列、第 2 行吃掉 $i + j$ 列——用 $j \in \{0, \pm 1\}$ 这个"错位"参数捕捉棋盘右侧的三种形状。关键洞察是"最优方案中不存在两艘水平船上下对齐"：如果第 1 行的 $(x - 1, x)$ 和第 2 行的 $(x - 1, x)$ 各放了一艘水平船，把它们替换成 $x - 1$ 列和 x 列的两艘垂直船，覆盖的格子完全相同、得分不变——所以水平船对齐永远不占优。这个观察把原本可能有 $O(n)$ 种错位量的形状压缩到 3 种。递推的三个分支对应"最右格怎么处理"：(a) $j = 0$ 时最右列两格都在，可以垂直放一艘盖住 $(i, 1), (i, 2)$ ，也可以选择"只吃掉最右格之一"——于是跳转到 $j = \pm 1$ 的错位状态；(b) $j = \pm 1$ 时只有一行多出一格，能做的就是水平船盖掉多出的那格和它左边的一格，或者留空。📖 讲义正文的解题步骤里直接写着："Guess how to cover the right-most square(s) in an optimal placement" (翻译：猜测最优放置中如何覆盖最右边的方格)——所有线性 DP (从 Fibonacci 到本题) 的 Relate 步本质上都是"猜第一个/最后一个元素怎么处理"。Base case 的 $2i + j < 2$ 对应"少于两格没法放船"，这个"凑不齐一艘船就归零"的边界处理你在 PS7 的 Pizza Partitioning 里也会见到。

[problem] Wafer Power (晶圆电源)

一家初创公司正在为高度并行计算设计新的电子电路。在一个圆形晶圆的圆周上均匀分布着 n 个 ports (端口)，每个端口要么接 power source (电源)，要么接 computing unit (计算单元)。每个计算单元需要来自电源的能量，能量通过蚀刻在晶圆顶面的 wire (导线) 在端口之间传输。然而，如果计算单元连接到一个太近电源，电源会过载并摧毁电路。此外，两条蚀刻的导线不得相互交叉。电路设计师需要一个自动化方法来评估不同设计的有效性，并请你帮忙。给定一个电源与计算单元在 n 个端口上的排列，描述一个 $O(n^3)$ 时间的 dynamic programming algorithm：通过在晶圆表面蚀刻非交叉的 wires 来把计算单元匹配到电源，使供电的计算单元数量最大化，

其中 wires 不得连接圆周上两个相邻的端口。下面是一个示例晶圆，用非交叉 wires 把计算单元（白色）连接到电源（黑色）。

示例晶圆示意（简化自原图，非精确还原）：圆周上 8 个端口逆时针排列为 $a_1(C), a_2(P), a_3(C), a_4(P), a_5(C), a_6(P), a_7(C), a_8(P)$ ，其中 C = computing unit（计算单元）、 P = power source（电源）；两条非交叉弦 $a_1 - a_4$ 与 $a_5 - a_8$ 分别连接计算单元与电源，且没有连接相邻端口。

[solution] Solution (解答) :

1. Subproblems (子问题)

- 令 $(a_1, \dots, a_n)$ 为绕晶圆逆时针循环排列的 ports, 其中 ports a_1 和 a_n 相邻
- 令 a_i 为 `True` (若该端口是 computing unit), 为 `False` (若是 power source)
- 想要匹配由非交叉 wires 连接的对立端口
- 若匹配穿过晶圆, 则需要独立匹配两侧的端口 (substrings (子串) !)
- $x(i, j)$: 限制在端口 a_k ($k \in \{i, \dots, j\}$) 时的最大匹配数
- 对 $i \in \{1, \dots, n\}$ 、 $j \in \{i - 1, \dots, n\}$
- $j - i + 1$ 是子串中的端口数 (允许 $j = i - 1$ 作为空子串)

2. Relate (关系)

- 猜测子串中第一个端口与哪个端口匹配。要么: – 第一个端口不与任何端口匹配, 尝试匹配其余部分; – 第一个端口与中间某个端口匹配, 尝试独立匹配两侧。
- 非相邻条件限制了 i 与某个端口 t 之间可能的匹配: – 若 $(i, j) = (1, n)$, 不能把 i 与最后一个端口 n 或 2 匹配, 所以尝试 $t \in \{3, \dots, n - 1\}$ – 否则只是不能把 i 与 $i + 1$ 匹配, 所以尝试 $t \in \{i + 2, \dots, j\}$
- 令 $m(i, j) = 1$ (若 $a_i \neq a_j$), $m(i, j) = 0$ (否则) (异类端口才匹配)

$$x(1, n) = \max(\{x(2, n)\} \cup \{m(1, t) + x(2, t - 1) + x(t + 1, n) \mid t \in \{3, \dots, n - 1\}\})$$

$$x(i, j) = \max(\{x(i + 1, j)\} \cup \{m(i, t) + x(i + 1, t - 1) + x(t + 1, j) \mid t \in \{i + 2, \dots, j\}\})$$

3. Topo (拓扑序)

- 子问题 $x(i, j)$ 只依赖严格更小的 $j - i$, 所以无环

4. Base (基本情况)

- 对 $j - i + 1 \in \{0, 1, 2\}$, $x(i, j) = 0$ (在 0、1 或 2 个相邻端口内无法匹配)

5. Original (原问题)

- 用递归 top-down (自顶向下) 或迭代 bottom-up (自底向上) 求解子问题。
- 原问题的解是 $x(1, n)$ 。
- 存储 parent pointers (父指针) 以重构匹配 (例如, 每步选择 t 或不匹配)。

6. Time (时间)

- 子问题数: $O(n^2)$
- 每个子问题的工作量: $O(n)$
- $O(n^3)$ 运行时间

[note] Note (译者注) : Wafer Power 是区间 DP (interval DP) 的经典形态, 和你在 Lecture 16 学过的 LCS、Lecture 17 学过的 Matrix Chain Multiplication (矩阵链乘) 同族: 子问题是一个连续区间 $[i, j]$, Relate 步枚举区间内的"分割点" t 。这里的关键是"若 a_i 与 a_t 匹配, 则区间被弦 (i, t) 切成互不相干的两半"——因为 wires 不得交叉, (i, t) 这条弦内部的端口只能互相匹配、外部的端口只能互相匹配, 于是拆成 $x(i + 1, t - 1)$ 和 $x(t + 1, j)$ 两个独立子问题。这与你学过的**括号匹配 (balanced parentheses) *DP 是同一个结构: 一条弦就像一对匹配的括号, 把问题递归地切成嵌套的子区间。为什么 t 的下界是 $i + 2$? 因为相邻端口不能连线, 所以 $t = i + 1$ 非法; 为什么 $(1, n)$ 特例中 t 还要排除 n ? 因为 a_1 与 a_n 在圆周上相邻。这个"循环序列拆成区间、区间首尾各有一个特例"的处理, 是环状问题的标准套路 (你在 PS7 的 Pizza Partitioning 里也见过 $\alpha_0 = \alpha_{2n}$ 的环形下标处理)。时间 $O(n^3)$ 来自 $O(n^2)$ 个区间 $\times$ 每个区间枚举 $O(n)$ 个 t 。📖 *讲义正文的表述: "If match across the wafer, need to independently match ports on either side (substrings!)" (翻译: 若匹配穿过晶圆, 则需要独立匹配两侧的端口 (子串!)) ——"穿过就分割成子串"是理解所有区间 DP 的钥匙。